Require: request r; the store S
Ensure: one uniform reply, after a send, a downgrade, a delay or a refusal

procedure PROCESSREQUEST(r)
  $p \leftarrow$ ATTEST(r); **if** $p = \bot$ **or** $r.\text{address} \in \mathcal{V}$ **then return** REFUSE()
  if $\neg$VALIDSESSION(r) **or** $n_{\text{sess}}[r.\text{session}] \geq 3$ **then return** REFUSE()
  if DENYLISTED(r) **then return** REFUSE()
  if $\neg$TAKE($r.\text{address}, r./24, r.\text{ASN}$) **then return** REFUSE()
  if $p = \text{web}$ **and** ($\text{score}(r) < \theta_w$ **or** $\text{origin}(r) \notin \mathcal{O}$) **then return** REFUSE()
  $(c, \text{live}, v) \leftarrow$ CHECKNUMBER($r.\text{number}$)
  if $\neg\text{live}$ **or** $c \in \{\text{off-list}, \text{premium}\}$ **then return** REFUSE()
  if $\neg$VALIDBODY(r) **or** $r.\text{number} \in \mathcal{X}$ **then return** REFUSE()
  if $(r.\text{bulk} \vee r.\text{test}) \wedge \neg r.\text{credential}$ **then return** REFUSE()
  $s \leftarrow$ the score of (1); $t \leftarrow$ TIER($s, v, r.\text{history}$)
  if $t = \text{block}$ **then return** REFUSE()
  if $t = \text{challenge}$ **then return** CHALLENGE()
  if $\neg$TAKE($\text{backoff}[r.\text{number}]$) **then return** REFUSE()
  if $\neg$TAKE($m \cdot K[r.\text{source}, p, \text{country}(r.\text{number})]$) **then return** REFUSE()
  $\text{mode} \leftarrow$ MODE(budget); **if** $\text{mode} = \text{emergency}$ **then return** REFUSE()
  $(\text{ok}, g) \leftarrow$ RESERVE($\text{count}, \text{spend}, \text{slot}[r.\text{block}]$)
  if $\neg\text{ok}$ **then return** REFUSE()
  $t \leftarrow$ RAISE($t, g, r.\text{history}$); **if** $t = \text{challenge}$ **then return** CHALLENGE()
  DISPATCH(r, t); WRITE(S)
  return REPLY()

procedure ONOUTCOME(x, o)
  for all $k \in \text{keys}(x)$ **do**
    $n_k \leftarrow n_k + 1$; $v_k \leftarrow v_k + [o = \text{verified}]$
    if $n_k \geq 50$ **and** $v_k < 0.1\, n_k$ **then** DENYLIST(k, 24 h)
  if $o = \text{verified}$ **then** $\text{trusted}[x.\text{number}] \leftarrow \text{true}$
  $b \leftarrow x.\text{block}$; $S_c[b] \leftarrow \max(-ch,\ S_c[b] + \delta_c(o))$
  if $o = \text{verified}$ **then** $S_s[b] \leftarrow \max(-ch,\ S_s[b] + \delta_s(x))$
  if $S_c[b] > h$ **or** $S_s[b] > h$ **then** VERDICT(b, 1 h); $S_c[b], S_s[b] \leftarrow 0$